AI Compiler Performance Engineering

Auto-Loops Books — ch30

Contents

1 Framework, Compiler, and Runtime Co-Design for Decode Goodput **1**

1.1 responsibility split 4

1.2 decode pipeline handoff 9

1.3 vllm yirage stack 12

1.4 mode selection fleet 16

1.5 production regression 19

1.6 Co-design decode benchmarking methodology 22

1.7 Key Takeaways 29

1.8 Conclusion 30

Bibliography **32**

Chapter 1

Framework, Compiler, and Runtime Co-Design for Decode Goodput

Production decode is a *three-way* contract: the inference **framework** schedules requests and owns KV paging; the **compiler** (YiRage middle-end + `superoptimize`) owns intra-step fusion and layout; the **runtime** (Persistent Kernel Layer 5) owns launches, device memory, and backend fallbacks (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022). When p99 latency stalls, teams that optimize only one leg recreate Chapter 1's framework fragmentation at warehouse scale (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Part VIII closes by making that contract explicit: every integration meeting draws the decode step **boundary** first, then assigns owners and counters (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016).

Chapter 28 mapped framework buckets and scheduler policy; Chapter 29 descended into YiRage Layer 5 Persistent Kernel runtime (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; NVIDIA, 2024; Goodfellow *et al.*, 2016). This chapter stitches the three parties into one production workflow: who owns prefill versus decode handoff, how vLLM paging composes with YiRage fused cores, which execution mode applies per subgraph on a heterogeneous fleet, and how regression

gates prevent silent drift across submodule bumps (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). The book's industrial arc ends here—not with another isolated kernel trick, but with deployable co-design anchored on decode goodput counters (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Readers should leave with a release train mental model. Framework teams ship scheduler and paging policy; compiler teams ship fused plans and manifest schemas; runtime teams ship PK launch descriptors and pool invariants (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016). No party optimizes in isolation: a framework upgrade that changes block table layout without bumping artifact cache salts breaks compiler handoff; a compiler win that ignores paging ABI wastes bytes per token; a runtime regression that leaks pool slots masquerades as scheduler overload (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Co-design means shared worksheets, shared request ids in traces, and shared rollback pins—not shared blame in postmortems (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Fregly mechanical sympathy at the three-way boundary treats each handoff as an API with semver, golden tests, and profiler evidence (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Dao *et al.*, 2022; NVIDIA, 2024; Goodfellow *et al.*, 2016). Chapter 12 execution modes reappear here as fleet policy: eager hulls for sampling branches, CUDA Graph buckets for launch-bound cells, MegaKernel and Persistent Kernel paths for byte-bound decoder cores (NVIDIA, 2024; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). DeepSeek-class disaggregation adds a fourth axis—prefill and decode pool routing—but the same boundary discipline applies inside each pool's decode step (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Three questions gate every co-design review before fleet promotion. First, does the responsible party for the moving counter actually own the lever being tuned (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Second, do handoff

manifests survive framework, compiler, and runtime version bumps without silent fallback (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Third, does triplet regression on GPU, CPU, and NPU backends show bytes per token and launches per token moving together or explained separately (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Goodfellow *et al.*, 2016). If any answer is no, pause feature work and fix the contract before tuning batch windows (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

This closing chapter is deliberately operational. Earlier parts taught rooflines, fusion legality, dialect lowering, and autotune search (Chen, 2020; Chen and YiRage Contributors, 2026; SemiAnalysis, 2024; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Part VIII asked where those ideas attach in servers humans operate (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016). The answer is never a wholesale framework replacement—it is a negotiated boundary with versioned handoffs and regression gates (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Teams that skip co-design and chase isolated kernel records on leaderboard shapes reproduce the fragmentation Chapter 1 diagnosed (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Co-design is the antidote measured in the same counters from prompt to token (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Warehouse-scale serving adds constraints single-GPU labs hide (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Continuous batching interleaves unrelated clients; cancel paths free slots unpredictably; disaggregated pools move KV across networks (Kwon *et al.*, 2023; Liu *et al.*, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Compiler and runtime optimizations must tolerate that chaos without leaking memory or violating paging ABI (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Framework optimizations must expose enough telemetry for compiler teams to see in-step counters, not only fleet aggregates (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016). Co-design is the discipline of making those telemetry paths first-class product requirements (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).



Figure 1.1: Responsibility split: framework, compiler, and runtime meet at the decode boundary; counters attach per party.

Leadership reviews should ask which party moved which counter last quarter, not which kernel microbenchmark improved (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Roadmaps that fund framework features without compiler handoff metadata or runtime descriptor tests recreate integration debt (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Conversely, compiler teams that ship artifacts without framework plugin support plan fail at the boundary (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Co-design funding follows counter movement with named owners (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). This chapter gives vocabulary for those staffing and budget conversations (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Part VIII therefore completes the narrative arc from mindset and hardware bounds through compiler stacks to deployable serving (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016). Readers who implement only one chapter should still implement this one if they operate fleets (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Theory without co-design contracts becomes shelfware; serving without counters becomes guesswork (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). The combined worksheet—boundary diagram, integration digest, mode map, triplet rows—is the minimum viable artifact set for production decode (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; NVIDIA, 2024; Goodfellow *et al.*, 2016).

1.1 responsibility split

Responsibility split is the operating table every production runbook must publish before on-call rotation (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026;

Table 1.1: Framework, compiler, and runtime responsibility split at the decode boundary.

Party	**Owns**	**Primary counter**
Framework	Batching, paging, routing, sampling policy, request lifecycle (Kwon *et al.*, 2023; Davies *et al.*, 2025)	Queue wait, fleet tokens/s, KV utilization
Compiler	Tiling, fusion, layout, legality, `superoptimize` artifacts (Chen and YiRage Contributors, 2026; Chen, 2020)	Bytes/token inside decode step
Runtime	PK launch, sync, pools, graph replay, backend registry (Chen and YiRage Contributors, 2026; NVIDIA, 2024)	Launches/token, warmup latency, pool high-water

Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022). Table 1.1 maps the three parties to concrete artifacts and counters. Ambiguity at the **boundary** between framework scheduling and compiler-owned decode cells is the dominant cause of misattributed regressions in fleet postmortems (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016).

The **framework** owns everything outside the decode step box: continuous batching, paged KV block tables, speculative decoding policy, disaggregated routing between prefill and decode pools, and client-facing SLO dashboards (Kwon *et al.*, 2023; Davies *et al.*, 2025; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016). Framework engineers optimize queue depth, slot assignment, and memory capacity—not softmax carrier layout inside a fused attention kernel (Davies *et al.*, 2025; Kwon *et al.*, 2023; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). When fleet tokens per second rise but bytes per token inside the step is flat, the framework did its job; the compiler and runtime did not (Chen, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

The **compiler** owns intra-step fusion and residency: which operators merge, which tensors stay on chip between layers, which layouts match paging gathers, and which plans are legal for graph capture (Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; SemiAnalysis, 2024; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016). Compiler output is not PyTorch source—it is versioned artifacts with fingerprints, manifest fields, and hardware registry ids from Chapter 29 (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Framework plugins consume those artifacts; they must not silently strip softmax carrier metadata or paging ABI version fields (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

The **runtime** executes compiler plans on device: Persistent Kernel loops, memory pool lifetimes, CUDA Graph capture buckets, backend fallbacks, and descriptor validation at each decode step (Chen and YiRage Contributors, 2026; NVIDIA, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Runtime engineers read nsys and pool accounting; they do not rewrite scheduler batching policy to mask launch storms (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). When launches per token regress after a runtime bump, bisect artifact digest and descriptor version before touching framework code (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016).

Draw the decode **boundary** on every architecture diagram: from slot mask and block table handle ingress through final hidden state before sampling (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Inside the box, compiler and runtime own bytes per token and launches per token; outside, framework owns queue time and paging efficiency (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Integration meetings that skip the boundary redraw recreate Chapter 1 fragmentation: compiler teams tune GEMMs while scheduler queue time dominates p99 (Davies *et al.*, 2025; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

On-call triage by party. Queue wait up, in-step ms flat: framework scheduling or capacity (Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). In-step bytes up, launches flat: compiler fusion or layout regression (Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow

et al., 2016). Launches up, bytes flat: runtime graph capture or PK descriptor regression (Vellaisamy *et al.*, 2026; NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). All three up: likely boundary mismatch—paging ABI changed without artifact invalidation (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Org design implications. Staff framework, compiler, and runtime teams with separate counter dashboards but shared release train (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016). Shared on-call without shared worksheets devolves into thrash: each party tunes its leg while p99 stalls (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Co-design standups read one row per request id: queue ms, in-step ms, launches, bytes, artifact digest (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Example 1.1.1. Regression after framework block size change: compiler cache hit loads stale PK plan—boundary contract requires cache salt bump on paging ABI change, not runtime knob tuning (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Industrial teams document the split in runbooks linked from fleet config repos (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). New hires should trace one decode token through framework slot assignment, compiler artifact load, runtime PK launch, and sampling egress before proposing optimizations (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016). That trace is the co-design literacy test for Part VIII (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Release trains should version the responsibility table alongside framework, compiler, and runtime artifacts (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). When a new feature spans two parties—for example speculative decoding in the framework plus extra PK launches in runtime—file a joint design doc with counter acceptance criteria before implementation (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016). Escalation paths should name DRI per party, not a

generic platform team that owns none of the levers (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Quarterly audits replay archived traces from peak traffic week and verify each party still matches the published table (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016).

Contract tests between parties catch boundary drift early (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Framework publishes paging ABI fixtures; compiler asserts superoptimize output loads against fixtures; runtime asserts PK descriptors accept fixture block tables (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Breaking any contract test blocks merge until semver bump and coordinated release notes ship (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). This is how co-design scales beyond a single heroic integration engineer (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Worked boundary exercise. Pick one production model and one fleet SKU (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). List ten operations in decode order from scheduler slot assignment through sampling (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Mark each operation framework, compiler, or runtime owned (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Attach one counter per party (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Run one request through traces and verify marks match reality (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Mismatches indicate undocumented boundary creep—fix the runbook before optimizing (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Repeat after every major framework or submodule bump (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Vendor and open-source upgrades flow through the same table (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). CUDA driver updates may change graph capture behavior; vLLM releases may change block tables; YiRage bumps may change manifest majors (NVIDIA, 2024; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026;

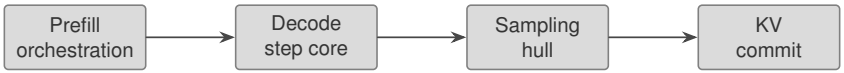


Figure 1.2: Decode pipeline handoff: prefill may stay framework-orchestrated; decode enters compiler-owned cores; sampling remains an eager hull outside capture.

Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Assign each upgrade type a default owner and regression template so on-call does not debate responsibility during incidents (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). The responsibility split is living documentation, not a one-time slide (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

1.2 decode pipeline handoff

Decode pipeline handoff sequences three phases with different owners: **prefill** ingestion, per-token **decode** compute, and **sampling** egress (Davies *et al.*, 2025; Dao *et al.*, 2022, 2023; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; AMD, 2024b,a; NVIDIA, 2022). Prefill often stays framework-orchestrated because shapes vary widely and batching policy dominates (Kwon *et al.*, 2023; Davies *et al.*, 2025; Dao *et al.*, 2022, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Decode steps enter compiler-owned cores when bytes per token and launches per token dominate at batch one (Chen, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Sampling and speculative branches remain *eager hulls* outside CUDA Graph capture per Chapter 12 legality rules (NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016).

Prefill handoff. Framework schedulers batch prompt tokens, allocate paging blocks, and may route long prompts to disaggregated prefill pools (Kwon *et al.*, 2023; Liu *et al.*, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Compiler involvement during prefill varies: some stacks superoptimize prefill profiles separately from decode; others reuse decode tiles inefficiently (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Dao *et al.*, 2022, 2023; Goodfellow *et al.*, 2016). Document which PK

mode the framework invokes at prefill-to-decode transition—mixing modes without manifest tags causes silent bandwidth waste (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Prefill completion must commit KV layout metadata the decode artifact expects: block table dimensions, head layout, and rope cache handles (Kwon *et al.*, 2023; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Decode handoff. Each decode iteration passes slot mask, page table pointer, step id, and hidden state buffers into the compiler-owned cell (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; NVIDIA, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Runtime validates descriptor version, launches PK or graph replay, and returns logits or hidden states to the framework (Chen and YiRage Contributors, 2026; NVIDIA, 2024; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Handoff latency includes framework Python overhead, plugin marshalling, and device sync—profile each separately before blaming fusion quality (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016). Continuous batching may swap slots mid-flight; decode artifacts must tolerate dynamic slot masks without re-capture on every step when graph mode is enabled (Kwon *et al.*, 2023; NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Goodfellow *et al.*, 2016).

Sampling handoff. Top-k, top-p, temperature, and stop criteria execute outside captured regions (NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Speculative decoding adds draft and verify loops that re-enter decode cores multiple times per user token (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016). Compiler plans must declare which tensors sampling reads and whether logits stay on device (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Framework policy changes to sampling should not require re-superoptimize unless carrier layout changes (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Handoff contracts deserve semver and golden-token tests at each phase boundary (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). CI should run prefill-only, decode-only, and full generate paths with shared artifact digests (Chen and YiRage Contributors, 2026;

Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016). Fail closed when manifest major version mismatches—never silently fall back to eager per-op decode (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

DeepSeek-class **disaggregated** stacks extend handoff across network boundaries: prefill pools write KV to remote decode pools (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Bytes per token then includes transfer cost; compiler and runtime optimizations inside decode pools still matter but must compose with transfer scheduling (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Profile end-to-end goodput, not in-pool ms alone (Davies *et al.*, 2025; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Example 1.2.1. Prefill completes at ctx=8192; decode PK warmed only to ctx=2048 bucket—tail latency spikes on long sessions until runtime bucket table matches framework max context policy (Kwon *et al.*, 2023; Chen, 2026; NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Multi-hardware delta. Prefill may use wide batch tiles on GPU while decode PK targets batch one; CPU and NPU backends need separate handoff manifests per Chapter 29 same-backend rule (AMD, 2024b,a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; NVIDIA, 2022; Dao *et al.*, 2022, 2023; NVIDIA, 2024; Goodfellow *et al.*, 2016).

Handoff latency budgets belong in SLO documents with separate allocations for framework marshalling, plugin sync, and device execution (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Teams that collapse these into one in-step millisecond metric misallocate optimization effort (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Log each segment with shared request id so postmortems identify whether regression entered at prefill commit, decode launch, or sampling return (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016). Long context sessions stress handoff most at prefill-to-decode transition when bucket tables must expand—warm expanded buckets during prefill completion, not on first decode token (Kwon *et al.*, 2023; Chen, 2026; NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*,

2016).

Timing diagram discipline. Draw prefill, decode, and sampling phases on one timeline per request class (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Mark where KV becomes visible to decode workers in disaggregated setups (Liu *et al.*, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Mark where logits cross back to framework for sampling (NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Ambiguous timing diagrams produce integrations that double-copy KV or synchronize too aggressively (Kwon *et al.*, 2023; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Review diagrams in design docs with the same rigor as API schemas (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Streaming responses add another handoff: partial tokens must egress while decode continues (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Framework streaming APIs should not force synchronous device sync each token unless product requires it (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Compiler and runtime plans should declare which steps tolerate async logits delivery (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Profile streaming workloads separately from batch benchmark harnesses (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Tail latency under streaming often traces to handoff sync policy, not fusion quality (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

1.3 vllm yirage stack

Vllm yirage stack is the reference **integration** pattern for Part VIII: vLLM continuous batching and paged KV plus YiRage fused decode core invoked each step (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022). Measure queue wait and in-step counters separately—fleet tokens per second can rise while launches per token inside the step stays at TaxBreak hundreds (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and

YiRage Contributors, 2026; Chen, 2020, 2026; Goodfellow *et al.*, 2016). The stack does not replace vLLM; it shrinks the decode cell vLLM calls (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Integration architecture. Typical flow: vLLM scheduler assigns slots and block tables; a custom attention or decoder backend loads YiRage artifact by digest at worker start; each decode step marshals descriptors into PersistentKernel (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; NVIDIA, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Plugin code lives at the framework boundary documented in Chapter 28 partial-export bucket (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; NVIDIA, 2024; Goodfellow *et al.*, 2016). Keep plugin surface minimal: slot mask, page table handle, step id, artifact digest—not raw tensor graphs per request (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

vLLM responsibilities retained. Paging, continuous batching, speculative decoding orchestration, and disaggregated routing stay in vLLM (Kwon *et al.*, 2023; Davies *et al.*, 2025; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). YiRage does not reimplement block allocators or queue policies (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). When KV utilization drops, tune framework batch windows before re-running superoptimize (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). When bytes per token inside the step stalls, escalate to compiler and runtime (Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

YiRage responsibilities absorbed. Fusion across attention, norm, and MLP seams; softmax carrier residency; PK launch collapse; graph capture buckets aligned to vLLM shape policies (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Chen, 2020; NVIDIA, 2024; Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Artifacts pin at worker boot from `deps/YiRage` submodule SHA recorded in fleet config (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Submodule bumps trigger regression triplet from Section 1.5 before traffic shift (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Integration testing template. Stand up vLLM worker with pinned artifact; run BS=1 decode at ctx 512, 2048, 8192; log queue ms, in-step ms, launches per

token, bytes per token (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Compare against vLLM plus FlashAttention baseline on same SKU (Dao *et al.*, 2022; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Promote only when both in-step counters improve or regressions are explained with profiler evidence (Davies *et al.*, 2025; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Run cancel and resume stress under continuous batching before fleet promotion (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Open components under `deps/DeepEP` and `deps/FlashMLA` illustrate MoE and MLA patterns that must compose with vLLM paging rather than bypass it (Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016). Integration docs should show where expert parallel all-to-all sits relative to PK entry—below scheduler, adjacent to decode cell (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Example 1.3.1. vLLM upgrade changes block table layout: YiRage cache hit serves stale gathers—integration contract requires artifact invalidation on paging ABI bump, not plugin hotfix alone (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Operational ownership. vLLM SREs own scheduler SLOs and paging capacity; YiRage owners own artifact pipeline and PK stability (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Shared dashboards correlate framework version, artifact digest, and runtime descriptor version per request id (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Integration failures at this boundary are the most common Part VIII incident class (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016).

Rollout playbooks for vLLM plus YiRage integration should mirror compiler artifact promotion: dev cluster golden tokens, staging canary at five percent traffic, production shift with rollback digest pinned (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016). Document which vLLM worker flags enable the custom backend and which remain framework defaults (Kwon *et al.*, 2023; Davies

et al., 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Training new operators on integration dashboards prevents conflating queue depth spikes with PK failures (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). When disaggregated prefill is enabled, verify YiRage artifacts on decode pool workers separately from prefill pool configs (Liu *et al.*, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Plugin minimization principle. The vLLM integration surface should stay thin enough to audit in one code review (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Heavy logic in plugins duplicates compiler and runtime responsibilities and forks across teams (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Push fusion and residency decisions into superoptimize artifacts; push launch stability into Persistent Kernel runtime (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016). Plugins marshal descriptors and handle error propagation only (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). When plugins grow past that scope, refactor boundary before next fleet promotion (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Capacity planning for integrated stacks counts both framework KV footprint and runtime pool high-water marks (Kwon *et al.*, 2023; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Scaling replicas without warming PK buckets on new nodes produces temporary tail spikes (Chen, 2026; NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Autoscalers should call runtime warmup hooks before marking pods ready (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Integration runbooks document expected warmup duration per artifact digest and context bucket (Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Ignoring warmup in capacity math causes false headroom during traffic ramps (Davies *et al.*, 2025; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

1.4 mode selection fleet

Mode selection fleet applies Chapter 12 execution modes at production scale: **eager** for debug and sampling hulls; **graph** for launch-bound decode cells; **megakernel** and Persistent Kernel paths for byte-bound fused cores—chosen per subgraph, not per monolithic binary (NVIDIA, 2024; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022). Fleet configs bucket context lengths, batch widths, and backend SKUs consistently so capture templates and PK warmup tables match scheduler policies (Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Eager mode. Use for development, numerics bisect, and any branch outside graph legality—sampling, dynamic control flow, debugging hooks (NVIDIA, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Eager baselines establish golden tokens for PK and graph paths (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Never ship eager per-op decode at scale when TaxBreak launch counts exceed fleet budget (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Graph mode. CUDA Graph replay removes host launch storms when fusion depth is insufficient to reach MegaKernel but launches per token dominate (Chen, 2026; NVIDIA, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Bucket graphs by shape signatures aligned to framework max batch and context policies (Kwon *et al.*, 2023; NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Goodfellow *et al.*, 2016). Graph mode without byte wins indicates need for compiler fusion, not more buckets (Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

MegaKernel and PK mode. Compiler-owned decode cores collapse layers inside Persistent Kernel loops, targeting bytes per token and residual launches (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Combine with graph capture where legality allows—PK provides fusion; graphs stabilize host overhead (Chen and YiRage Contributors, 2026; NVIDIA, 2024; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Profile carrier lifetimes when bytes per token stall despite PK adoption (Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

Fleet bucketing policy. Document which mode applies to each subgraph in fleet config: prefill profile, decode profile, sampling hull, MoE communication (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Goodfellow *et al.*, 2016). Heterogeneous fleets maintain parallel artifact lines per hardware registry id from Chapter 29 (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024b,a; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Never reuse Hopper PK binaries on Blackwell without re-search (Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2022; SemiAnalysis, 2024; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; Goodfellow *et al.*, 2016).

Mode selection meetings should attach counter worksheets: if launches per token move but bytes do not, stop adding graph buckets and fix fusion (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). If bytes move but queue wait dominates p99, return to framework scheduling (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Fleet-wide mode policy prevents per-team improvisation that breaks artifact compatibility (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Multi-hardware delta. Graph mode is CUDA-centric; XDNA fleets use static AIE binaries; CPU fleets use OpenMP-blocked eager or PK host loops (AMD, 2024b,a; NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Same fleet config schema should encode backend-specific mode enums (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024a,b; Goodfellow *et al.*, 2016).

Example 1.4.1. Team enables graph capture on entire decoder including sampling—illegal capture causes silent fallback to eager launches; mode map must exclude sampling hull (NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016).

Fleet configuration management should treat mode maps as first-class config objects with review gates (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; NVIDIA, 2024; Goodfellow *et al.*, 2016). Changing a subgraph from graph to PK mode without re-warming pools causes cold-start p99 spikes (Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Autoscaling policies must account for warmup time when mode maps include capture-heavy paths (Chen, 2026; NVIDIA, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Document mode map version in artifact manifest so bisect correlates performance changes with policy edits (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Multi-region fleets replicate mode maps only after triplet passes in each region's SKU mix (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016).

Debug versus production mode policy. Development clusters may run eager decode for fast iteration (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Production clusters must enforce mode maps checked into config management (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024; Goodfellow *et al.*, 2016). Accidental eager fallback in production reads as cost overrun and tail latency inflation (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026; Goodfellow *et al.*, 2016). Alert when launch counts exceed eager thresholds on production endpoints (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Separate debug mode toggles from feature flags so marketing demos do not become silent production configs (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Hybrid mode maps are normal: prefill eager or wide-tile, decode PK plus graph, sampling eager hull, MoE communication custom (NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016). Document each subgraph mode in fleet config with owner and counter expectation (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Changing one subgraph mode without re-running triplet on affected rows invites silent regressions (Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023;

Goodfellow *et al.*, 2016). Mode maps should be diff-reviewed like code (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024; Goodfellow *et al.*, 2016).

SRE teams should store mode map version in incident tickets alongside framework and artifact versions (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Replaying incidents without mode context wastes hours tuning the wrong subgraph (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Quarterly drills rotate mode map edits through staging triplet before any production diff (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Fleet mode policy is co-design infrastructure as critical as paging block size (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024; Goodfellow *et al.*, 2016).

1.5 production regression

Production regression gates fleet promotion on triplet counter suites: bytes per token, launches per token, and framework-level tokens per second—measured on GPU, CPU, and NPU backends where applicable (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024b,a; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022, 2024; Goodfellow *et al.*, 2016). Version pins for `deps/YiRage` submodule, framework release, and artifact digest must move together through canary deploys (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Submodule bumps require re-run of search caches and PK warmup profiles before traffic shift (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Triplet worksheet. Row one: launches per token on warmed PK at BS=1 for ctx 512, 2048, 8192 (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Row two: bytes per token on same buckets (Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Row three: end-to-end tokens per second including queue wait at representative fleet load (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Sign off only when each row improves or regressions carry profiler evidence and owner (Davies *et al.*, 2025; Chen, 2026; Chen

and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Regression triggers. Framework paging ABI change; compiler manifest major bump; runtime descriptor version change; hardware registry id addition; CUDA driver or toolkit upgrade (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; NVIDIA, 2024; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026; AMD, 2024b,a; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Each trigger reruns triplet on primary SKU plus one secondary backend in CI matrix (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024a,b; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016).

Canary and rollback. Shift artifact digest on fraction of replicas; compare triplet dashboards against control cohort (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016). Rollback redeploys prior artifact bundle—not framework scheduler alone (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Store rollback pins beside fleet config for one-click revert (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Bytes-focused regression. Memory-Floor style suites catch HBM traffic regressions graph replay hides (Chen, 2026; NVIDIA, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Pair byte counters with carrier lifetime profiles when fusion changes (Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Silent staging copies from paging integration show as byte regressions with unchanged launch counts (Chen, 2026; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

CI pipelines pin submodule SHA, compile native extensions, run superoptimize smoke, PK decode ten tokens, and compare golden hash (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Extend smoke with vLLM integration template from Section 1.3 before production promotion (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Record worksheet JSON in artifact

store for audit (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Example 1.5.1. Submodule bump passes import test but fails PK golden hash—regression gate blocks merge; fleet stays on prior digest until triplet reruns green (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Long-horizon **regression** debt accumulates when teams skip triplet on minor framework patches (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Schedule weekly automated reruns on pinned production configs even without code changes—driver drift and thermal throttling move counters (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Treat regression suites as production infrastructure, not optional research scripts (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Cross-team regression review meetings compare triplet trends across framework, compiler, and runtime release notes (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016). A flat launches per token row with rising bytes per token row signals compiler or paging integration regression (Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Rising queue wait with flat in-step rows signals framework capacity or scheduling regression (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Archive worksheet JSON for each production promotion to enable quarter-over-quarter audits (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Regressions without archived worksheets are not reproducible and should not close incident tickets (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Closing the book loop. Return to Chapter 1 counters with co-design eyes (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). TaxBreak launches per token, Memory-Floor bytes per token, and framework tokens per second now have owners and handoffs (Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*,

2016). MegaKernel craft from Part IV–V lives inside compiler artifacts (Chen, 2020; Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Compiler theory from Part VI–VII lives in manifest schemas and legality metadata (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Runtime execution from Chapter 29 lives in PK pools and graph buckets (Chen and YiRage Contributors, 2026; NVIDIA, 2024; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Framework policy from Chapter 28 lives outside the decode boundary (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). The book ends where production begins: pinned digests, triplet gates, and shared traces (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016).

Regression culture. Treat triplet failures as cross-team signals, not blame events (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016). Framework teams publish paging ABI changelogs; compiler teams publish manifest semver notes; runtime teams publish descriptor diffs (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). On-call runbooks link each changelog entry to required regression reruns (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). When all three parties follow the rhythm, co-design becomes routine operations instead of heroic firefighting (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016). That operational maturity is the practical definition of decode goodput at warehouse scale (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

1.6 Co-design decode benchmarking methodology

Benchmarking three-way co-design extends Chapter 1 counters with party-owned ledgers: framework queue wait and fleet tokens per second, compiler bytes per token inside the decode boundary, runtime launches per token and pool high-water marks (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2022). Build a decode cell with BS=1, ctx buckets

Metric	Co-design party owner
Queue wait / fleet t/s	Framework scheduler
Bytes/token (in-step)	Compiler fusion plan
Launches/token	Runtime PK + graph replay
Artifact digest match	Compiler + runtime handoff
Mode map version	Fleet policy (Section 1.4)
Paging ABI semver	Framework + compiler contract

Table 1.2: Co-design decode counter worksheet (closes Part VIII).

$\{512, 2048, 8192\}$, pinned framework release, `deps/YiRage` SHA, artifact digest, mode map version, and paging ABI from Chapter 28 (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Warm-up must include framework slot assignment, compiler artifact load, runtime PK construction, graph capture steady state, and at least one thousand decode steps—microbenches that skip the boundary misattribute wins to the wrong party (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; NVIDIA, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Report queue ms, in-step ms, launches per token, and bytes per token on the same dashboard row (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024).

Example 1.6.1. Co-design gate: warmed decode at ctx=2048 must show in-step bytes per token within 10% of Memory-Floor worksheet, launches per token within 5% of TaxBreak logs, and queue wait flat versus baseline at representative fleet load—attach artifact digest, mode map hash, and request-id trace slice (Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Boundary replay discipline. Run three worksheets on identical traffic: framework-only eager hull, compiler artifact through PK microbench, and full vLLM–YiRage plugin path from Section 1.3 (Kwon *et al.*, 2023; Chen and YiRage Contributors,

2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Deltas between worksheet two and three isolate framework scheduler tax; deltas between one and two isolate compiler residency (Davies *et al.*, 2025; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Never promote co-design releases without all three rows green or explained (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Handoff manifest pinning. Nightly CI must load the same manifest bundle as staging: paging ABI version, artifact fingerprint, descriptor semver, submodule SHA, and mode map digest (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Drift between compiler search clusters and serving replicas produces bimodal latency histograms that no single-party tuning fixes (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024).

Triplet backend matrix. Extend co-design benchmarks to GPU, CPU, and NPU registry ids where fleets are heterogeneous (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024b,a; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Framework paging policy may differ per backend, but in-step byte and launch counters must use identical ctx buckets for cross-SKU comparison (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024).

Mode map interaction benchmarks. Hybrid fleet configs mix eager sampling hulls, CUDA Graph decode buckets, and PK MegaKernel cores (NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Liu *et al.*, 2024; Dao *et al.*, 2022; AMD,

2024b,a; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Benchmark each subgraph mode independently before composing the fleet map—graph capture on sampling branches fails silently and inflates launches per token (NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016).

Regression bisect by party. When p99 stalls, sequence: (1) diff artifact digest across replicas; (2) compare mode map version; (3) replay boundary trace on golden request id; (4) diff bytes per token worksheet; (5) diff launches per token; (6) compare queue wait at fleet load (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Skipping boundary replay keeps teams in multi-week blame loops (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Organizational metrics. Align OKRs with party-owned counters—framework teams own queue wait and fleet tokens per second, compiler teams own in-step bytes per token, runtime teams own launches per token and pool high-water (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). PRs that move one counter without worksheet evidence fail co-design review (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Canary and rollback worksheets. Shift artifact digest on a fraction of replicas; compare all three party rows against control cohort (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Rollback redeploys prior artifact bundle and mode map—not framework scheduler knobs alone (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Documentation contract. Publish co-design benchmark artifacts with every fleet promotion: boundary diagram, three-row worksheet, manifest pins, mode map diff, and triplet backend summary (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Without published ledgers, partners cannot replay co-design regressions (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Release checklist. Before tagging a co-design release, require green boundary replay, manifest match, mode map triplet pass, and counter movement on all ctx buckets (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Partial green is how production inherits mixed schedules behind one load balancer (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Long-context co-design stress. Extend benchmarks to ctx=16384 when paging allows; framework block growth and compiler gather layouts must move bytes per token together (Kwon *et al.*, 2023; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Fail CI when queue wait rises without in-step counter movement—likely boundary ABI drift (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Speculative decode co-design. Draft-verify stacks span framework policy and dual runtime PK instances (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Benchmark acceptance-rate sweeps with separate worksheets per model and shared request-id traces (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD,

2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024).

Efficiency metric. Track *co-design-attributed decode efficiency*: fleet tokens per second divided by roofline tokens per second after subtracting unexplained queue wait and attributing in-step bytes and launches to named parties (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Publish party numerators weekly in the same dashboard (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

CI golden triple policy. Store framework baseline, PK microbench, and full plugin scripts side by side; any PR that moves bytes per token or launches per token in the wrong direction fails before merge (Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Golden triples prevent silent operator-first relapses at the boundary (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Disaggregated pool benchmarks. Prefill and decode pools route KV across networks; co-design worksheets must tag which pool owns each counter row (Liu *et al.*, 2024; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Network transfer bytes are framework-owned; in-step fusion bytes remain compiler-owned—conflating them hides regressions (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Postmortem artifact schema. Store JSON per incident with request id, queue ms, in-step ms, launches per token, bytes per token, artifact digest, mode map version, and paging ABI semver (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*,

2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Incidents without schema fields should not close until worksheets are archived in the fleet artifact store (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Executive readout template. One-page promotion summaries show three party counter deltas side by side—executives approve co-design releases, not isolated kernel names (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Hide counter regressions behind fleet p50 charts and teams rediscover Chapter 1 fragmentation (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Final gate sentence. Do not merge co-design changes without green golden triple, manifest match, and mode map diff reviewed by framework, compiler, and runtime DRIs on the same pinned artifact hash (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Anything less is costly three-way drift wearing co-design labels in production fleets (Chen, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Closing synthesis. Co-design benchmarking is complete when framework, compiler, and runtime worksheets agree within published tolerances on three ctx buckets with manifest and mode map pinned—otherwise keep the release in staging (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). That discipline closes the book loop back to Chapter 1 counters with named owners and shared rollback pins (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Nightly cadence. Run golden triple scripts nightly on pinned GPU CI runners with fixed seeds; replay through framework plugin weekly and triplet backends

monthly when hardware pools allow (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Flaky tests that skip manifest assertions teach teams to ignore critical co-design release gates in every weekly production release train cadence (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Worksheet archive policy. Version co-design worksheets beside compiler fingerprints and mode maps so bisect compares party-owned counters across releases without re-running ad hoc scripts (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Archive JSON schema fields match postmortem templates so on-call and promotion reviews share one authoritative source of truth for fleet operators and on-call engineers (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

1.7 Key Takeaways

- **Three-way co-design.** Framework, compiler, and runtime each own distinct counters; optimize the leg that moves your worksheet row (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026, 2020; Goodfellow *et al.*, 2016).
- **Boundary is API.** Decode step boundary, paging ABI, capture buckets, and PK entry points documented with semver (Kwon *et al.*, 2023; NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025).
- **vLLM plus YiRage integration.** Scheduler and paging stay framework-side; fused decode core shrinks in-step bytes and launches (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026).
- **Map execution mode per subgraph, not per binary.** Eager, graph-captured, and megakernel/Persistent-Kernel modes should be chosen per decode cell, because a single binary-wide mode is wrong for some subgraphs

(NVIDIA, 2024; Chen and YiRage Contributors, 2026). A dynamic-shape preprocessing step may stay eager while the fused decoder layer runs as a captured PK, and forcing one mode everywhere leaves goodput on the table (Vellaisamy *et al.*, 2026; Chen, 2026).

- **Triplet regression gates.** Bytes per token, launches per token, tokens per second on pinned submodule and artifact digest (Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).
- **Pin the submodule and smoke-test in CI.** Pin `deps/YiRage` to a digest and run a Persistent-Kernel decode smoke test, since an import-only check passes while the native co-designed stack is silently broken (Chen and YiRage Contributors, 2026; Chen, 2020). The three-way co-design only holds if the framework, compiler, and runtime versions are tested together against the same pinned artifact (Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

1.8 Conclusion

This chapter closes Part VIII and the book's industrial arc: hardware constraints (Part II–III), MegaKernel craft (Part IV–V), compiler theory and stacks (Part VI), production tuning (Part VII), and now *deployable* framework–compiler–runtime cooperation anchored on YiRage and decode goodput counters (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Chapter 26 production runbooks and Chapter 27 trend baselines supply fleet pins and paging ABI expectations; this chapter adds triplet regression gates and mode maps on top (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016). Draw the decode boundary, assign framework/compiler/runtime owners, integrate vLLM paging with YiRage artifacts, map execution modes per subgraph on your fleet, and gate every promotion on triplet regression (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; NVIDIA, 2024; Goodfellow *et al.*, 2016). Future work tracks autonomous search from Chapter 25 and new hardware entries in `HardwareRegistry`—always profiled on bytes per token and launches per token at batch one, never on FLOPs alone (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026, 2020; Goodfellow *et al.*, 2016).

The loop closes where it opened: mechanical sympathy measured in counters your fleet can audit (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016).

Before you close the repository, run one end-to-end worksheet on your target SKU: warmed Persistent Kernel, three context buckets, vLLM or equivalent scheduler at representative load, triplet rows logged, artifact digest pinned in config (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). If that worksheet passes review, Part VIII is not theory—it is your production baseline (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). If it fails, return to the section whose counter moved and fix the contract before shipping features (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016). Co-design is maintenance, not a one-time integration project (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Share the worksheet JSON with adjacent teams so framework, compiler, and runtime on-call engineers interpret the same numbers during incidents (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016). That shared artifact closes the book loop in operational practice, not only in prose (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Thirty chapters of mechanical sympathy converge on this worksheet—run it once on hardware you ship (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Archive the passing run in your fleet config repo as proof Part VIII is live (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Bibliography

AMD (2024a). Ai engine technology. `https://www.AMD.com/en/products/adaptive-socs-and-fpgas/technologies/AI-engine.html`. 1, 3, 5, 9, 11, 12, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30

AMD (2024b). AMD XDNA architecture. `https://www.AMD.com/en/technologies/XDNA.html`. 1, 3, 5, 9, 11, 12, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30

Chen, J. (2026). Memory-bound but not bandwidth-limited: The physical AI inference gap in batch-1 LLM decode. *arXiv preprint arXiv:2605.30571*. 44-cell cross-GPU study; CUDA Graphs A/B on H100 and L4. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31

Chen, X. (2020). Compiler-based hardware profiling for software-defined architecture. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31

Chen, X. and YiRage Contributors (2026). Yirage: Multi-backend LLM inference optimization. `https://github.com/chenxingqiang/YiRage`. Yield Revolutionary AGile Engine; extends Mirage with CUDA, CPU, MPS, Triton, NKI, and MLIR backends. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31

Dao, T. *et al.* (2023). Flash-decoding for long-context inference. PyTorch blog and Stanford CRFM technical report. Parallelizes attention along KV sequence; up to 8× decode speedup at long context. 1, 5, 9, 11, 12, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30

Dao, T., Fu, D., Ermon, S., Rudra, A., and Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*. 1, 2, 5, 6, 9, 10, 11, 12, 13, 14, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31

Davies, M. *et al.* (2025). Liminal: Exploring the frontiers of LLM decode performance. *arXiv preprint arXiv:2507.14397*. Limit study; DeepSeek-V3 provisions 10× more

decode than prefill nodes. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31

Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. *Proceedings of the 29th Symposium on Operating Systems Principles*. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31

Liu, A. *et al.* (2024). Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*. 1, 2, 3, 5, 9, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30

NVIDIA (2022). Nvidia hopper architecture in-depth. NVIDIA Technical Blog. 1, 5, 9, 11, 12, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30

NVIDIA (2024). Cuda graphs. CUDA Programming Guide. 1, 2, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31

SemiAnalysis (2024). Nvidia tensor core evolution: From volta to blackwell. Newsletter. 1, 2, 3, 5, 6, 9, 11, 12, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30

Vellaisamy, P., Tripathi, S., Natarajan, V., Thenarasu, S. S., Blanton, S., and Shen, J. P. (2026). Taxbreak: Unmasking the hidden costs of LLM inference through overhead decomposition. *arXiv preprint arXiv:2603.12465*. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31